

Storybook: documentación de funcionamiento, cobertura y pruebas

Estado del catálogo de componentes documentado con Storybook, cómo está configurado, cómo ejecutarlo, qué se verificó y qué queda pendiente.

✓ Build, typecheck, lint y tests en verde

Proyecto: ordenamiento (rama `manuel`)

Fecha del reporte: 1 de septiembre de 2026

Versión de Storybook: 10.5.10 (`@storybook/html-vite`)

Componentes documentados: 21 · Historias: 60

1. Qué es Storybook en este proyecto

Storybook es el catálogo visual de componentes de la interfaz: permite renderizar cada pieza de UI de forma aislada (sin backend, sin sesión, sin base de datos) con distintas combinaciones de props para inspeccionarla, documentarla y detectar regresiones visuales antes de que lleguen a una página real.

La particularidad de este proyecto es que **no usa React**: el runtime de UI es `remix/ui`, propio del framework Remix 3 sobre el que está construida la app (`(handle: Handle<Props>) => renderFn`), montado con `createRoot(container).render(...)` directamente sobre el DOM). Por eso el proyecto usa el renderer `@storybook/html-vite` (HTML puro) en vez del renderer de React de Storybook, y necesita un pequeño puente propio para poder montar componentes de `remix/ui` dentro del canvas de Storybook.

2. Configuración

Toda la configuración vive en `.storybook/`:

Archivo	Responsabilidad
<code>main.ts</code>	Declara el patrón de descubrimiento de historias (<code>app/**/*.stories.@(ts tsx)</code>), el addon de documentación, el framework <code>@storybook/html-vite</code> , y ajusta Vite: activa JSX automático con <code>jsxImportSource: 'remix/ui'</code> y vacía <code>process.env</code> en el build de navegador (varios módulos del servidor leen <code>process.env.*</code> a nivel de módulo; en el navegador <code>process</code> no existe).
<code>preview.ts</code>	Carga recursos globales que los componentes institucionales dan por hecho: la tipografía Montserrat, el web component de Iconify (para <code><iconify-icon></code> e <code>app/ui/admin/icon.tsx</code>), y la hoja de estilos de Leaflet (para los mapas). Define además los fondos de <code>backgrounds</code> con los tokens de color institucionales (Civic Horizon) y un reset básico del body del iframe.
<code>remix-root.ts</code>	El puente real: expone <code>mountRemix(renderElement)</code> , que envuelve cualquier componente de <code>remix/ui</code> en una función <code>render(args) => HTMLElement</code> que Storybook sabe ejecutar — crea un contenedor <code><div></code> , monta el componente ahí con <code>createRoot</code> , y lo devuelve.

Todas las historias del proyecto siguen el mismo patrón de wiring: importan `mountRemix`, definen un `meta` con `title` namespaced (`UI/`, `Login/`, `Admin/`, `POETDUM/`, `Público/`, `Participation/`) y exportan una o más `StoryObj` con distintos `args`.

3. Cómo ejecutarlo

```
npm run storybook      # servidor de desarrollo, http://localhost:6006
npm run build-storybook # build estático de producción del catálogo
```

No requiere backend, base de datos ni variables de entorno adicionales: es un entorno 100% de navegador sobre datos de ejemplo definidos en cada historia.

4. Cobertura: qué está documentado

21 COMPONENTES	22 ARCHIVOS .STORIES.TSX	60 HISTORIAS (VARIANTES)	6 NAMESPACES
-------------------	--------------------------------	--------------------------------	-----------------

UI/ — sistema de diseño base

Componente	Archivo	Historias
Button	app/ui/button.stories.tsx	12 (variantes, tamaños, estado deshabilitado, como enlace, ancho completo)
NavBar	app/ui/nav-bar.stories.tsx	1
Form/Field	app/ui/form/field.stories.tsx	6 (incluye TextArea y CheckboxField como historias adicionales)
Form/DireccionFields	app/ui/form/direccion-fields.stories.tsx	3 (civic, con errores, admin)

Login/

Componente	Archivo	Historias
LoginForm	app/ui/login/login-form.stories.tsx	2
LoginHeader	app/ui/login/login-header.stories.tsx	1
LoginFooter	app/ui/login/login-footer.stories.tsx	1
TextField	app/ui/login/text-field.stories.tsx	3 (email, password, con error)
LoginAlert nuevo	app/ui/login/login-alert.stories.tsx	4 (error, success, warning, info)

Admin/

Componente	Archivo	Historias
Icon	app/ui/admin/icon.stories.tsx	4
MapaSelector nuevo	app/actions/admin/public/mapa-selector.stories.tsx	2 (vacío, con coordenadas)

POETDUM/

Componente	Archivo	Historias
Mapa	app/actions/poetdum/public/mapa.stories.tsx	1
SesionesSection nuevo	.../sections/sesiones.stories.tsx	2 (con datos, vacío)

ActividadesSection nuevo	<code>.../sections/actividades.stories.tsx</code>	3 (próximas, realizadas, vacío)
DescargasSection nuevo	<code>.../sections/descargas.stories.tsx</code>	1
DocumentosSection nuevo	<code>.../sections/documentos.stories.tsx</code>	3 (con datos, filtrado, vacío)
SeguimientoSection nuevo	<code>.../sections/seguimiento.stories.tsx</code>	2 (con datos, vacío)

Público/ y Participation/

Componente	Archivo	Historias
PromptButton	<code>app/actions/public/prompt-button.stories.tsx</code>	1
ParticipationForm nuevo	<code>.../participation/participation-form.stories.tsx</code>	3 (vacío, con valores previos, con errores)
SubmitButton nuevo	<code>.../participation/public/submit-button.stories.tsx</code>	1
SuccessDialog nuevo	<code>.../participation/success-dialog.stories.tsx</code>	2 (con folio, sin folio)
UploadField nuevo	<code>.../participation/upload-field.stories.tsx</code>	2 (vacío, con error)

5. Deliberadamente fuera de cobertura

No todo componente del código recibió una historia; estas exclusiones fueron evaluadas explícitamente, no son omisiones:

Archivo	Motivo
<code>app/ui/admin/admin-layout.tsx</code>	Envuelve <code><Document></code> , que renderiza un <code><html><head><body></code> completo con scripts del pipeline de assets del servidor. Montarlo dentro del <code><div></code> del canvas de Storybook es inválido sin antes separar el shell interno del layout de página completa.
<code>app/components/Button/Button.tsx</code>	Componente duplicado sin ninguna referencia en el código (confirmado por grep) — parece código muerto abandonado del <code>Button</code> real en <code>app/ui/button.tsx</code> .
<code>app/styles/colors.tsx</code> , <code>app/styles/shadows.tsx</code>	Son tokens de estilo, no componentes visuales.
<code>success-dialog-boost.tsx</code>	Script <code>clientEntry</code> que solo monta un <code></code> para abrir el diálogo — sin salida visual propia; ya se ejercita dentro de la historia de <code>SuccessDialog</code> .

Todas las <code>*-page.tsx</code> y <code>*-controller.tsx</code>	Son route-level: dependen de loaders/actions/middleware del servidor. Consistente con el patrón ya existente en el repo antes de esta tarea.
---	---

6. Verificación y pruebas realizadas

Antes de dar por cerrada la cobertura, se corrió la batería de verificación completa del proyecto — todo en verde:

Comando	Resultado
<code>npm run typecheck</code>	OK sin errores de tipos
<code>npm run lint</code>	OK sin advertencias de ESLint
<code>npm run format:check</code>	OK en los archivos nuevos (las 54 advertencias del repo son preexistentes, no relacionadas con esta tarea)
<code>npm test</code>	OK 203 pruebas pasando, 1 fallo esperado (preexistente)
<code>npx storybook build</code>	OK build de producción exitoso, incluye los 22 módulos de historias

Pipeline de revisión de 4 agentes

Siguiendo el flujo de trabajo definido en `AGENTS.md` para cambios no triviales, la cobertura de Storybook pasó por las cuatro fases:

1. **Developer** — implementó las 11 historias nuevas siguiendo el patrón ya establecido.
2. **Architect** — aprobó la ubicación de archivos, confirmó la ausencia de duplicación, y validó las exclusiones deliberadas. Detectó y se corrigió un espacio en blanco accidental preexistente en `actividades.tsx` sin relación con las historias.
3. **QA** — re-corrió typecheck/lint/tests/format/build de forma independiente y confirmó que los `args` de cada historia coinciden con las interfaces reales de cada componente (no solo que compilan). Aprobó sin observaciones.
4. **Testing** — intentó romper el resultado: confirmó que el fallback sin red del selector de mapas es real (no solo un comentario optimista), que no hay `any` ni `mix` mal formado, y que las combinaciones de datos de ejemplo son coherentes con lo que ocurre en producción. Sin bloqueantes.

Adicionalmente se ejecutó una **revisión de seguridad** específica sobre el diff completo de la rama (56 archivos, no solo Storybook): sin hallazgos de inyección, path traversal, XSS, SSRF ni exposición de datos sensibles. El framework de sanitización ya existente (`isSafeCssColor` , `isSafeImageUrl` , `sanitizeText`) no fue alterado.

7. Pendientes conocidos

Tema	Detalle	Prioridad
Fuga de instancias de Leaflet entre historias	<code>mountRemix</code> nunca llama <code>root.dispose()</code> al cambiar de historia o mover un control — Storybook reemplaza el DOM directamente	Media

	(<code>canvasElement.innerHTML = ''</code>) sin pasar por el ciclo de vida de <code>remix/ui</code> . Afecta a <code>Mapa</code> y al nuevo <code>MapaSelector</code> : cada cambio de historia deja un <code>L.map()</code> huérfano con sus listeners. Es un problema de la infraestructura del puente (<code>remix-root.ts</code>), preexistente a esta tarea, no de los componentes ni del código de producción.	
Dependencia de CDN en historias con mapa	<code>Mapa</code> y <code>MapaSelector</code> cargan Leaflet desde <code>unpkg.com</code> en tiempo de ejecución del navegador. Sin red, el contenedor y el ciclo de vida igual se pueden probar, pero el mapa no se pinta. Comportamiento esperado y documentado en el propio código.	Informativo
<code>AdminLayout</code> sin historia	Requiere primero separar el shell de navegación/sidebar del <code><Document></code> de página completa antes de poder documentarlo de forma segura en Storybook.	Media
<code>app/components/Button/Button.tsx</code>	Código muerto, sin referencias, viola la convención del proyecto de evitar <code>app/components/</code> . Candidato a eliminar en una tarea de limpieza aparte.	Baja

8. Recomendaciones

- Cerrar la fuga de `mountRemix` guardando el `root` anterior en un closure y llamando `.dispose()` antes de crear uno nuevo — beneficia a las dos historias con Leaflet y a cualquier historia futura con estado interno o listeners.
- Antes de dar historia a `AdminLayout` , extraer su contenido en un componente de "shell" que no dependa de `<Document>` , dejando el `<Document>` solo para el nivel de página real.
- Eliminar `app/components/Button/Button.tsx` en una tarea de limpieza dedicada, confirmando primero (ya hecho) que no tiene referencias.
- Mantener el patrón actual (namespace + `mountRemix` + args realistas tomados de los tipos reales) para cualquier componente nuevo que se agregue al proyecto.